Aya Burpee, Brinda Moudgalya
ID Numbers: 2462847, 2337294
EE/CSE 371
October 31, 2025
Lab3 Report

Design Procedure

In this lab, we created circuits that interact with the DE1_SoC audio CODEC interface to record and play back audio. In Task1, we developed the logic for passing audio input from the microphone to the output speakers when the CODEC is ready to write and read data. In Task2, we played a static, single tone from a .mif file, using a ROM that read out the values, to the audio CODEC, and created a top-level module that can toggle between playing input audio from the microphone or playing audio from the values stored in the ROM with SW[9] of the board.

In Task3, we made a FIR filter that can filter the audio from the microphone or the ROM, then created another top level module that has the functionality of the task2 top level module with SW[9] but has the additional functionality of being able to play the filtered vs unfiltered audio from either task by toggling SW[8].

Task 1:

Task 1 was completed by simply passing the readdata_left and readdata_right values that were outputted from the audio CODEC back to the CODEC through the writedata_left and writedata_right signals respectively, and setting the read and write signals to go high when both the read_ready and write_ready signals from the CODEC are true. This was all done within the starter kit top level module for part 1 that included the clock generator, audio CODEC interface, and the audio/video configuration.

Task 2:

For task 2, we first created a separate module that included the control circuit portion of playing audio from the ROM, in which we instantiated the ROM module from Quartus and logic for incrementing the read address. Developing the logic of looping back to the first address of the ROM after reaching the last address was simple enough; the difficult part of this was figuring out the timing for when we should increment the address. If we increment the address every CLOCK_50 cycle, because the audio CODEC's sampling rate is 48kHz, there would be many

values in between each sampling of the CODEC that wouldn't be read, leading to an unexpected output. We needed to increment the read address at the same rate as the sampling frequency. Here's how we dealt with the timing:

Instead of using a counter that creates a pulse every 1042 cycles of CLOCK_50, we emulated a 48kHz clock by using the handshake signals between the audio CODEC and the control circuit.

The drawback of the 48kHz counter is that can't exactly get 48kHz because 50 million doesn't divide evenly into 48k. 48kHz would be a pulse every 1041.6666… cycles of CLOCK_50, so even if we made a counter, we wouldn't be able to match the sampling frequency exactly.

The logic we use for emulating 48kHz without a counter is by creating a signal that is true only when read_ready and write_ready are both high AND at least one of them has a positive edge. This signal pulses for 1 clock cycle at a time at a frequency of around 48kHz, and we use it as an enable signal to drive operations that need to match the sampling frequency of the CODEC(incrementing the read address of the ROM, inputting new data into the filter). The positive edge detection makes it so that even when an overall module is running on CLOCK_50, new data is only outputted for one CLOCK_50 cycle within the many cycles at which read_ready and write_ready are both high, matching the pace of the sapling frequency. In addition, detecting the positive edge for at least one of the signals (read_ready, write_ready) means that even if read_ready and write_ready are not completely in sync, the enable will pulse if one of the signals is already high and the other signal goes high afterwards.

After testing the task 2 circuit, we created a top-level module for instantiating the task 2 circuit and added the logic for switching between playing audio from the ROM when SW[9] is on and playing audio from the microphone(like in task 1) when SW[9] is off.

Task 3:

For task 3, we first created a FIR filter module, starting out by mapping out the inner workings by hand to figure out how the filter actually works. In the FIR, we instantiated the FIFO buffer module given to us for homework 3, with a singular change: we set the default values inside the FIFO to zero. This was to remove the risk of running into issues of performing arithmetic operations with undefined values in the adder or accumulator within the filter. For the rd and wr inputs of the FIFO buffer, we used the same enable signal used for incrementing the address in circuit 2 so that the FIFO would be driven at the same rate as the sampling frequency, when there is new input data. We made sure that the accumulator was also driven by the same

enable signal so that the FIFO and the accumulator were in sync. After testing the filter in ModelSim, we created the final top-level module containing all components from tasks 1 and 2, plus two FIR filters for filtering each of the left and right audio channels. We also added logic for playing out the filtered audio of the chosen task (based on SW[9]), when SW[8] is high, and playing the unfiltered audio when SW[8] is low.

Our final top-level diagram, circuit 2 diagram, and FIR filter diagrams are shown below.

Top level:

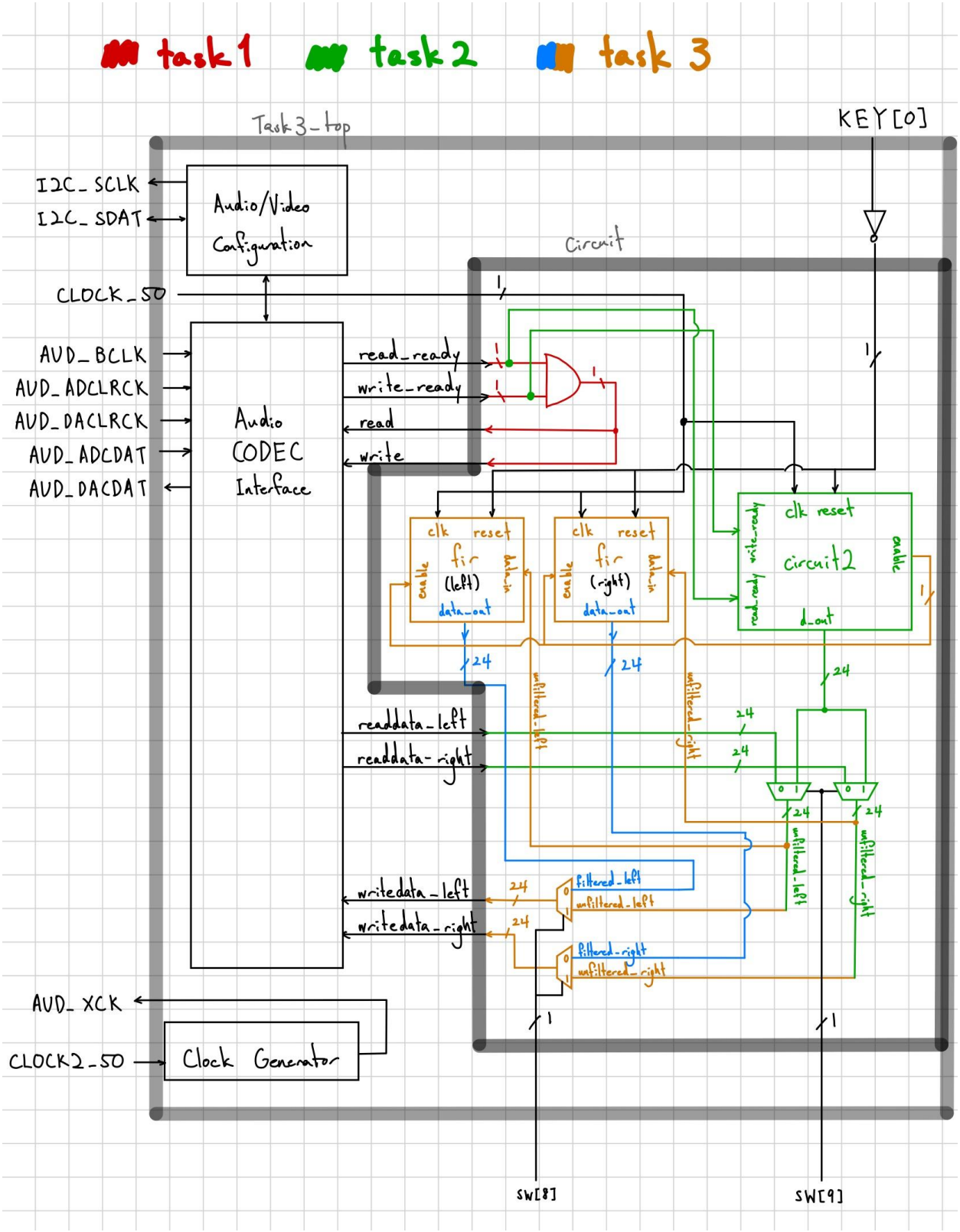


Figure 1: Task 3 top level diagram

Circuit 2:

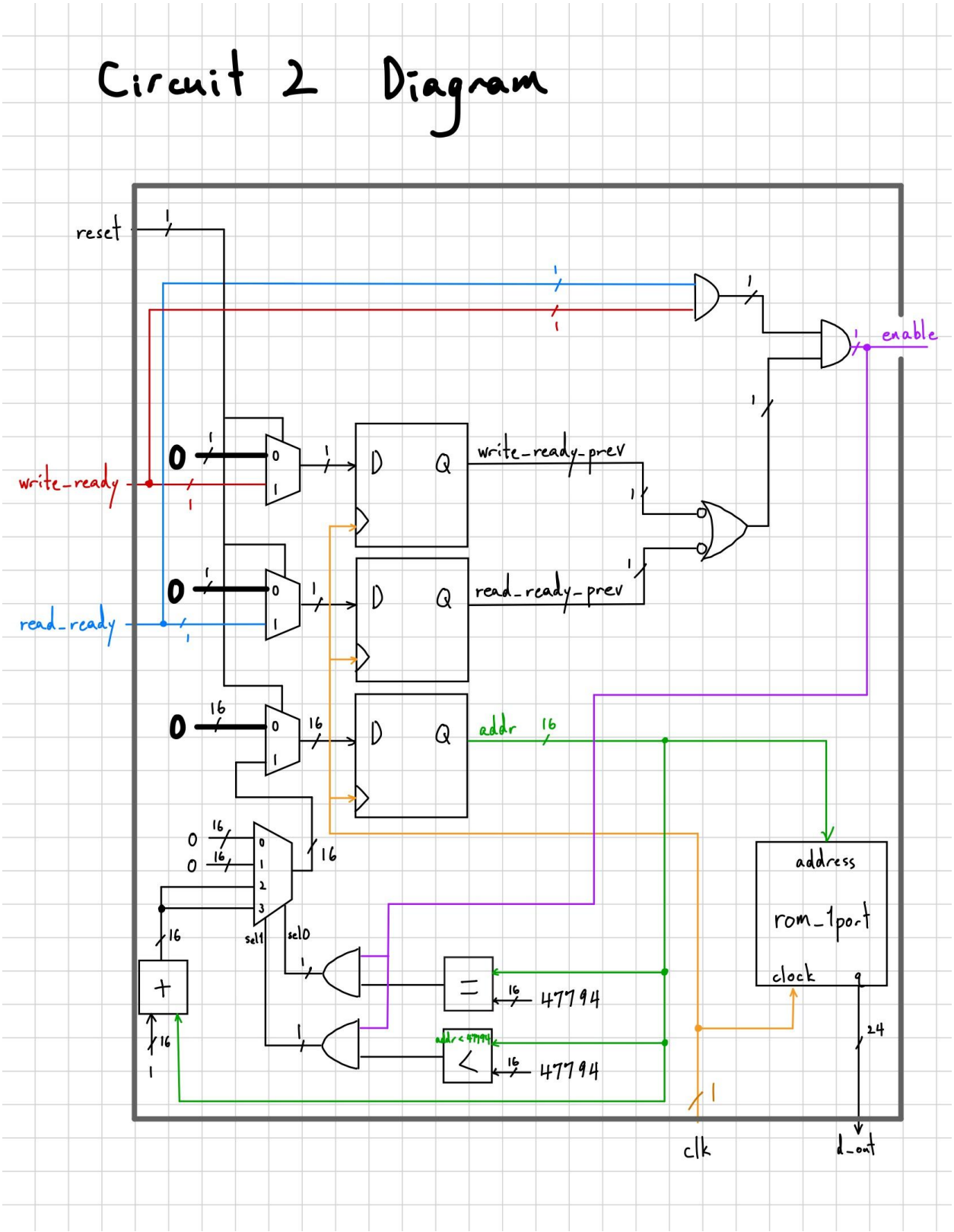


Figure 2: Circuit 2 diagram

FIR:

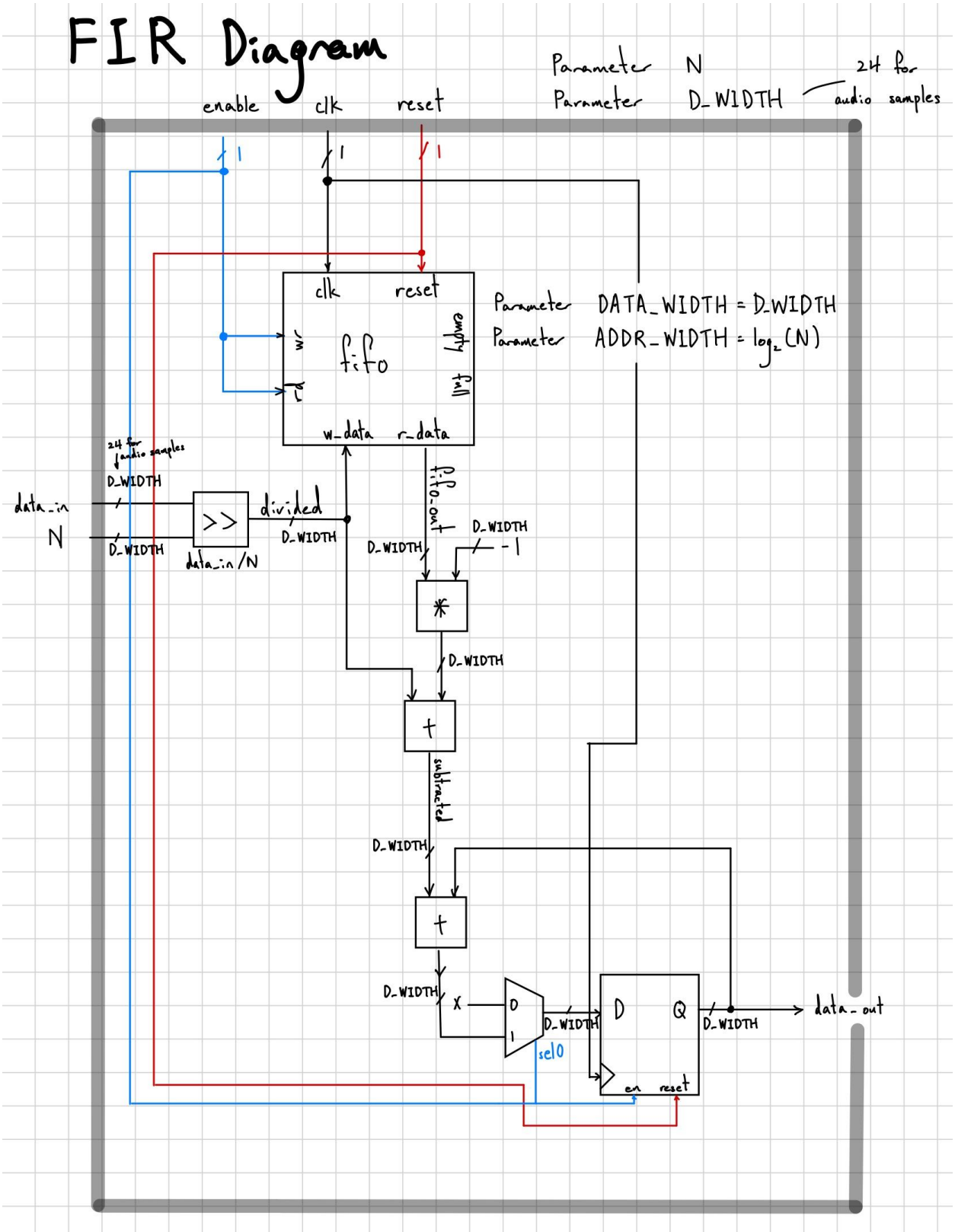


Figure 3: FIR diagram

Results:

We were able to create a system that passes audio through the FPGA when SW[9] is low, play a tone from memory when SW[9] is high, and filters the audio when SW[8] is high. When we loaded the final top-level module and played the audio in LabsLand, the filtered audio (of both the microphone input and from the ROM) sounded quieter and clearer. We tested the filter in LabsLand with different buffer sizes, and found that with small buffer sizes (<= 32), there was little to no difference in the filtered and unfiltered audio. With the buffer size at around 128 or 256, the filtered audio had a noticeably clearer tone and was slightly quieter than the unfiltered audio. With large buffer sizes (>=1048), the audio was again more clear; however the volume decreased drastically. Therefore we determined that the “sweet spot” for the buffer size is 128 to 256. Although there was a limited amount of testing we could do in ModelSim, we did test our task 2 circuit and FIR filter in ModelSim before instantiating them in the top-level modules. The simulation results are shown below.

Circuit 2 testing:

As mentioned above when discussing timing, the ROM address (addr) should only increment when read_ready and write_ready are both high AND at least one of them has a positive edge. (this is when the enable goes high) This makes it so that the address increments at around 48kHz, the sampling frequency, even if the read_ready and write_ready signals from the CODEC are slightly offset. (also explained above)

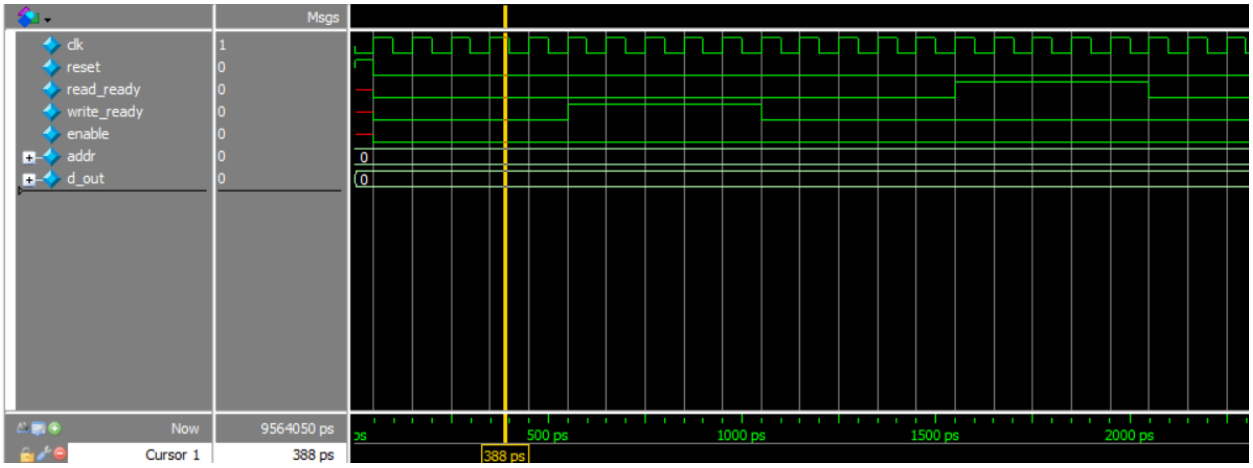


Figure 4: circuit 2 non-incrementing conditions

From above: We can see that even when read_ready or write_ready have a positive edge, the address doesn't increment since read_ready and write_ready both need to be high as well.

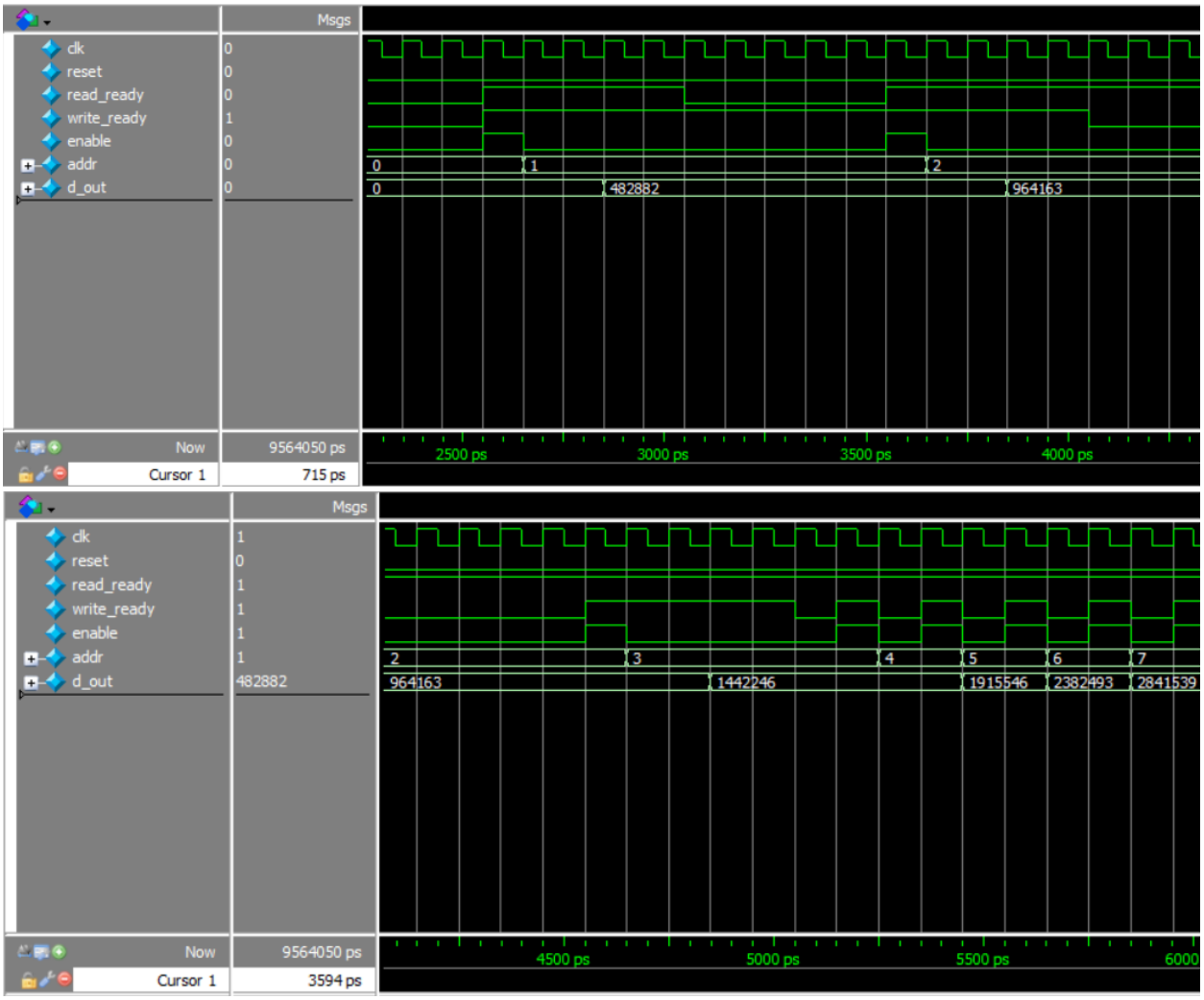


Figure 5: circuit 2 address incrementing

When read_ready and write_ready go high at the same time, the enable is driven high, and the address increments the next clock cycle. Same with when write_ready is already high and read_ready has a positive edge, or when read_ready is already high and write_ready has a positive edge.

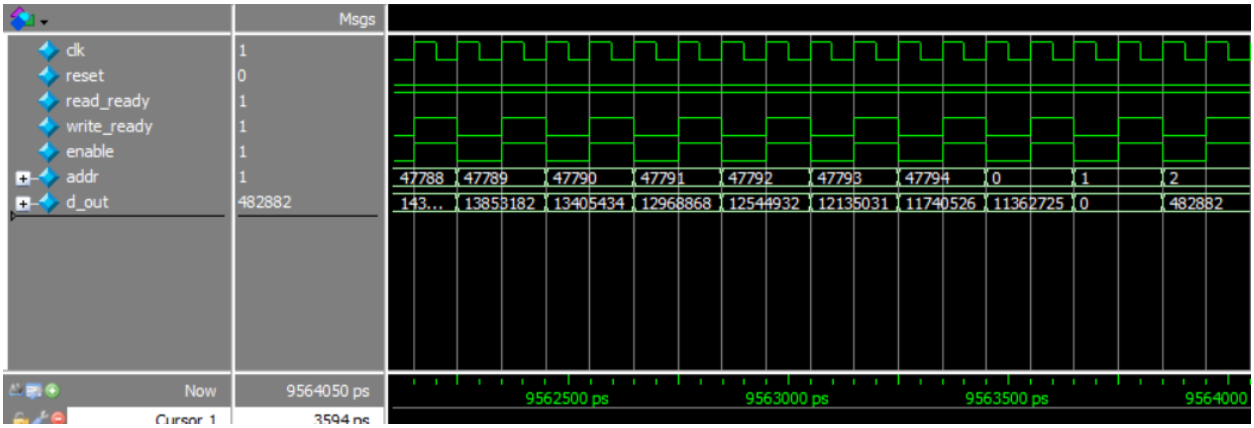


Figure 6: Circuit 2 address loop back

We see that after reaching the last address(47794), the address loops back to 0 without hiccups when enable is asserted.

We had initially generated a .mif file that stores data for 48000 address. However, when testing task2 in LabsLand, we noticed a hiccup in the sound when the ROM reached the last address and looped back to 0. This was caused by the fact that the sound wave was discontinuous as it looped back to the beginning. To fix this, we found the very last address at which the audio sample had a value of 0, and deleted everything after it so that when looping back to address 0, there would be a continuous sinusoidal wave. In our case, the last address we chose to keep in the .mif file was 47794. We also updated the values in the conditionals in circuit 2 of incrementing the address to match the last address number.

FIR filter testing:

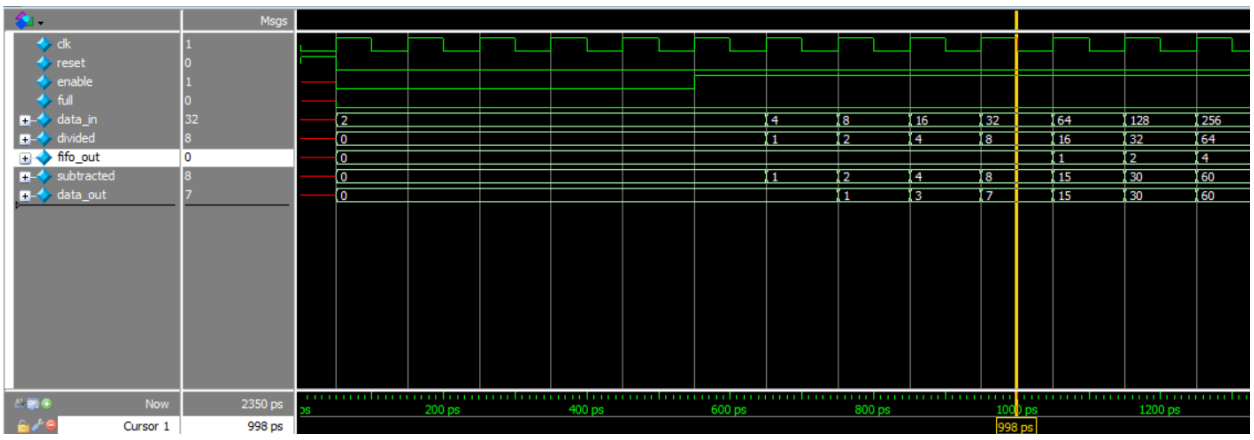


Figure 7: FIR filter testing -- first half

In the filter, the “divided” value is equal to data_in / N (size of the FIFO buffer), where N is 4 in this case. We can see that this is consistent with the waveforms. The FIFO in the filter writes in

and reads out values simultaneously. Since the FIFO buffer is preloaded with zeros, the output of the is going to be 0 for N cycles, then start cycling through the input values (“divided”) from N cycles previously, which we see: Once the FIFO is enabled, fifo_out stays at zero for 4 clock cycles, then starts cycling through the “divided” values from 4 cycles before. “subtracted” is the result of subtracting “divided” by fifo_out, which we also see is consistent with the waveforms. Finally, data_out is the output of the filter, which is equal to the sum of the previous data_out value and “subtracted”, which is also consistent with what we see in the simulation.

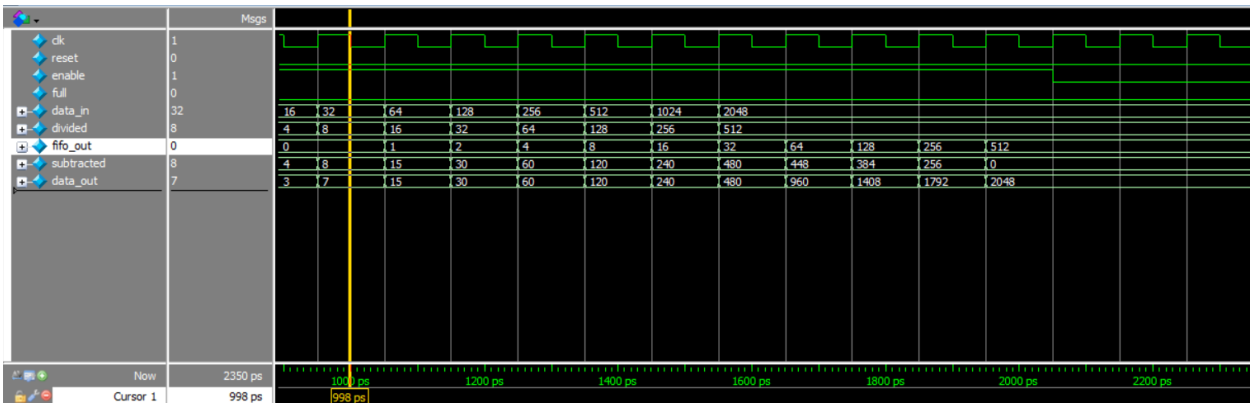


Figure 8: FIR testing -- second half

At the end of the testbench, we continuously feed in the same value(2048) into the filter for N cycles(4 cycles). Since the average value after the 4 cycles would be the same as data_in, we expect data_out to be 2048 after 4 cycles, which is exactly what we see.

Fifo testbench:

We used the fifo buffer code provided in hw3. The only thing we changed was that in the register file, we preloaded the array register with zeros so that all outputs would be defined, as explained above.

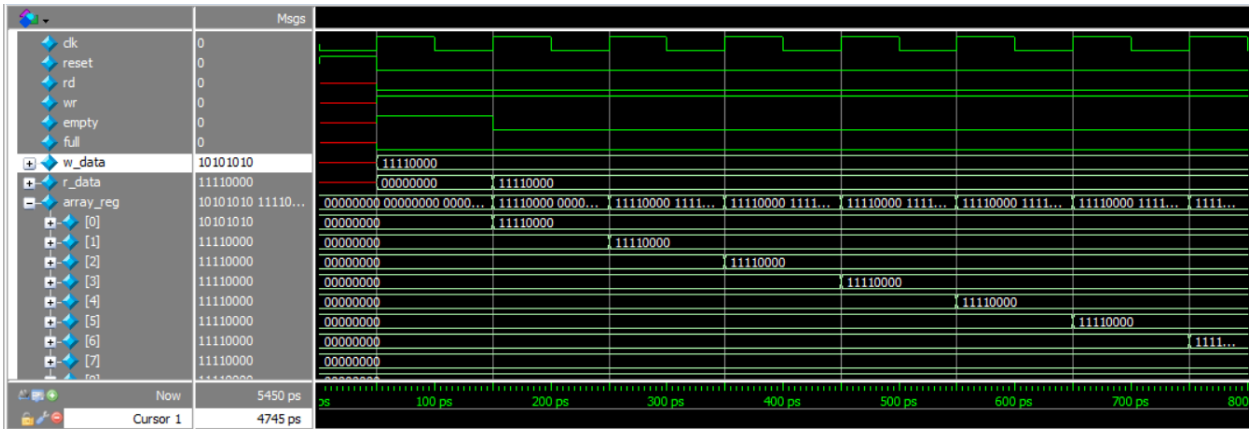


Figure 9: FIFO testing -- default values

As expected, we see that array_reg stores zeros from the beginning.

The flow summary of the final task 3 top level module is shown below:

Flow Summary

<<Filter>>

Flow Status	Successful - Mon Oct 27 12:37:38 2025
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	DE1_SoC
Top-level Entity Name	task3_top
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	N/A
Total registers	511
Total pins	21
Total virtual pins	0
Total block memory bits	1,176,576
Total DSP Blocks	0
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	1
Total DLLs	0

Figure 10: Task 3 top level flow summary

Experience Report

In terms of the coding, this lab had a moderate difficulty, mostly from implementing our FIR filter. Overall, this lab was more frustrating than anything. We spent tens of hours trying to test our code in LabsLand, because for the first week and a half of the lab, all microphones and around two thirds of the speakers were not working, and we would have to keep synthesizing and queueing for a board that actually worked. We also spent many, many hours trying to get rid of the pulsing in our audio that we mentioned in the last paragraph of our circuit 2 testing section. We had assumed that it was a problem with our code and that somehow our timing

was not implemented correctly, and spent lots of time tweaking our timing, until we figured out that it was a wrap-around discontinuity with the mif file.

The lab took approximately 45 hours, broken down as follows:

- Reading and interpreting lab specs for the tasks 1 and 2 – 1 hour
- Implementing tasks 1 and 2 – 3 hours
- Testing and debugging task 2 with ModelSim – 1 hour
- Reading and interpreting lab specs for task #3 + tracing the FIR – 2 hours
- Implementing task 3 – 1 hour
- Testing and debugging the FIR filter with ModelSim – 2 hours
- Testing the top-level modules of each task in LabsLand – 10 hours
- Figuring out the enable timings to match the sampling frequency, including trying to implement the design using a 48kHz signal enabling counter – 10 hours
- Trying to fix the pulsing audio in task 2 – 10 hours
- Drawing block diagrams – 5 hours
- Writing report – 2 hours